Homework 4: Generating Cats Images with GANs

Due Date April 14, 2026
Total Points 50

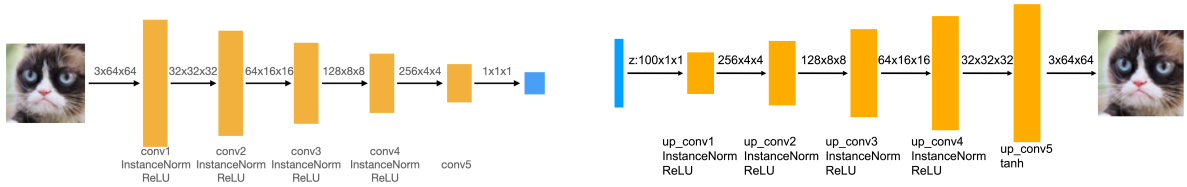
START HERE: Instructions

Collaboration policy: All are encouraged to work together BUT you must do your own work (code and write up). If you work with someone, please include their name in your write-up and cite any code that has been discussed. If we find highly identical write-ups or code or lack of proper accreditation of collaborators, we will take action according to strict university policies. See the Academic Integrity Section detailed in the initial lecture for more information.

Submitting your work:

- Your submission has two parts: (i) a LaTeX written PDF report that details all the requested deliverables (without any code!!) and (ii) the code for the assignment (as .py files only). You will upload the PDF report in gradescope, marking which parts of the report corresponds to which deliverable. The code will be uploaded separately, and we will run "software similarity" software on it to detect violations of the integrity policy.
- You can discuss this HW with others, but you are **not** allowed to share, borrow, copy or look at each other's codes.
- All code that you submit must be yours (except for the starter code that we provide). Specifically, unless specified otherwise, you are **not** allowed to use any software packages or code from the internet or other resources, except for `numpy`, `torch`, `torchvision`, `scipy`, `PIL`, `matplotlib`, and other built-in packages such as `math`.

NOTE: Partial points will be given for implementing parts of the homework even if you don't get the mentioned numbers as long as you include partial results in this pdf.



(a) Discriminator (b) Generator

Figure 1: DCGAN model architecture. (a) Architecture of the discriminator. (b) Architecture of the generator.

In this assignment, you will explore hands-on experience in coding and training Generative Adversarial Networks (GANs) Goodfellow et al. [2014]. This homework consists of two parts: the first part includes the implementation of Deep Convolutional GAN (DCGAN) Radford et al. [2016] and training it to generate grumpy cats images; the second part explores different architectures and training objectives of DCGAN; In both parts, you will gain experience implementing GANs by writing code for the generator, discriminator, and training loop, for each model. In all parts of this homework, you will gain experience implementing GANs, designing its architecture and objectives, and training loop of GANs.

1 Part 1: Deep Convolutional GAN (30 points)

For the first part of this assignment, we will implement a slightly modified version of Deep Convolutional GAN (DCGAN). A DCGAN is simply a GAN that uses a convolutional neural network as the discriminator, and a network composed of transposed convolutions as the generator. **In the assignment, instead of using transposed convolutions, we will be using a combination of an upsampling layer and a convolutional layer to replace transposed convolutions.** To implement the DCGAN, we need to specify three things: 1) the generator, 2) the discriminator, and 3) the training procedure. We will develop each of these three components in the following subsections.

1.1 Implement the Discriminator (5 Points)

The (modified) discriminator of DCGAN consists of a series of convolutional layers, batch/instance normalization layers, and ReLU activation function, as shown in Fig. 1(a),

Implement this architecture by filling in the *init* function and *forward method* of the *DCDiscriminator* class in models.py, shown below. The *conv_dim* argument does not need to be changed unless you are using larger images, as it should specify the initial image size.

1.2 Implement the Generator (5 Points)

Now, we will implement the generator of the DCGAN, which consists of a sequence of upsample+convolutional layers that progressively upsample the input noise sample to generate a fake image. The generator in this DCGAN has the architecture as shown in Fig. 1(b).

Implement this architecture by filling in the *init* and *forward method* of the DCGenerator class in models.py. Note: Use the *up_conv* function (analogous to the conv function used for the discriminator above) in your generator implementation. We find that for the first layer (*up_conv1*) it is better to directly apply convolution layer without any upsampling to get 4x4 output. To do so, you'll need to think about what your kernel and padding size should be in this case. Feel free to use *up_conv* for the rest of the layers.

1.3 DCGAN Training Loop (20 Points)

Next, you will implement the training loop for the DCGAN. A DCGAN is simply a GAN with a specific type of generator and discriminator; thus, we train it in exactly the same way as a standard GAN. The pseudo-code for the training procedure is shown in Algorithm 1. The actual implementation is simpler than it may seem from the pseudo-code: this will give you practice in translating math to code.

1.3.1 Implementation (5 Point)

Fill the *training_loop* part in vanilla_ganpy. There are 4 numbered bullets in the code to fill in for the discriminator and 3 bullets for the generator. Each of these can be done in a single line of code, although you will not lose marks for using multiple lines.

Algorithm 1 DCGAN algorithm

Draw m training samples $\{x_0, x_1, ..., x_m\}$ from data distribution p_{data}.
Draw m noise samples $\{z_0, z_1, ..., z_m\}$ from noise distribution p_z.
Generate fake images from the noise: $G(z_i)$ for $i \in \{1, ..., m\}$.
Compute the discriminator loss.
Update the parameters of the discriminator.
Draw m new noise samples $\{z_0, z_1, ..., z_m\}$ from noise distribution p_z.
Generate fake images from the noise: $G(z_i)$ for $i \in \{1, ..., m\}$.
Compute the generator loss.
Update the parameters of the generator.

DCGAN will perform poorly without data augmentation on a small-sized dataset because the discriminator can easily overfit to a real dataset. To rescue, we need to add some data augmentation such as random crop and random horizontal flip. You need to fill in advanced version of data augmentation in data_loaderṗy. We provide some script for you to begin with. You need to compose them into a transform object which is passed to CustomDataset.

1.3.2 Training and Visualization (15 Point)

Train DCGAN by running the completed *vanilla_ganṗy*. Provide your training hyper-parameters. Show a grid of generated cats images using basic augmentation and advanced augmentation for the beginning of training (400 iterations) and the end of training (6000 iterations) here respectively. Also plot the training loss correspondingly. (In total 6 images are expected to shown here.)

To get full grade for advanced, we expect your model to at least perform visually better than vanilla GAN, some example signs we will be looking at includes, less noise, less mode collapse (i.e., the ability to generate different images), presents more cat features.

2 Part 2: Architectures and Objective Functions (20 points)

In this section, we will try different architecture of the generator and discriminator in DCGAN and different loss functions to train model for generating cats images with better visual quality. Some of these are new to you, and are not covered in class, so we expect you to read the original paper, and understand how to implement it. TAs will also be at office hours to help.

For this section, you must implement from scratch in PyTorch, this means you must not use any function that trivialize the implementation, including but not limited to torch.nn.utils.spectral_norm, torchvision.models.resnet18, etc.

- Spectral Norm [Miyato et al., 2018]
- Wasserstein GAN [Arjovsky et al., 2017] with gradient penalty [Gulrajani et al., 2017]
- LSGAN [Mao et al., 2017]
- Custom Architecture - Implement a new GAN discriminator and generator with your custom architecture, feel free to take inspiration from any models covered or not covered during lecture.

For each of the above item, write a brief summary of your implementation/approach, and show a grid of generated cat images at the end of training (6000 iterations).

References

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks, 2016.

Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks, 2018. URL `https://arxiv.org/abs/1802.05957`.

Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein gan, 2017.

Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans, 2017. URL https://arxiv.org/abs/1704.00028.

Xudong Mao, Qing Li, Haoran Xie, Raymond Y. K. Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks, 2017.